ResearchLens Viva Voce Questions & Answers

Project Overview & Objectives

Q1: What is ResearchLens and what problem does it solve in academic research?

> ResearchLens is an AI-powered research paper analysis platform that automates the literature review process. It solves the problem of information overload — researchers spend weeks manually reading hundreds of papers to extract themes, identify gaps, detect trends, and synthesize findings. ResearchLens analyzes papers in minutes, extracting structured insights through 10 specialized modules. This accelerates research planning and hypothesis formation.

Q2: Explain the end-to-end workflow of analyzing research papers with ResearchLens.

> 1. User uploads PDF research papers via the frontend dashboard
> 2. Backend extracts text/metadata using PDF parser + spaCy NLP preprocessing
> 3. Module 1 (Summarization): Condenses each paper to key points
> 4. Module 2 (Topic Modeling): Uses BERTopic to identify research themes across corpus
> 5. Module 3 (Gap Detection): Analyzes coverage gaps using LLM prompting
> 6. Module 4 (Trend Detection): Identifies emerging research directions
> 7. Module 5 (Visualization): Creates maps and graphs of research landscape
> 8. Module 6 (RAG Chatbot): Allows Q&A using Ollama + retrieved context
> 9. Module 10 (Scientific Honesty): Assesses bias and reproducibility flags
> 10. Results saved to MongoDB; user can export reports or ask follow-up questions

---

Q3: What are the key modules in ResearchLens and their primary functions?

> 1. **Summarization**: Auto-generates key takeaways per paper
> 2. **Topic Modeling**: Clusters papers into research themes using BERTopic
> 3. **Gap Detection**: Identifies underexplored research areas
> 4. **Trend Detection**: Ranks emerging research directions by recency/frequency
> 5. **Visualization**: Creates interactive knowledge maps and correlation graphs
> 6. **RAG Chatbot**: Q&A system powered by Ollama LLM + document context
> 7. **Scientific Honesty**: Assesses bias, conflicts of interest, reproducibility flags

---

Q4: Why is automation important in literature review and research analysis?

> Manual literature review is time-consuming (weeks/months), error-prone, and not scalable. Researchers often miss important papers or fail to connect cross-domain insights. Automation enables:
>
> - **Speed**: Analyze 100+ papers in minutes vs. weeks
> - **Consistency**: Same criteria applied to all papers
> - **Comprehensiveness**: No papers missed due to fatigue
> - **Pattern discovery**: Trends and gaps visible at scale
> - **Reproducibility**: Documented analysis methods for peer review

---

Technical Architecture

Q5: Describe the full tech stack: frontend, backend, databases, and external services.

> - **Frontend**: React 18 + TypeScript + Tailwind CSS + Vite (fast HMR)
> - **Backend**: Express.js (Node.js) + middleware (auth, logging)
> - **Database**: MongoDB (document storage for papers, analyses, user accounts)
> - **NLP/Python**: spaCy (tokenization/NER), BERTopic (topic modeling), torch/transformers (embeddings)
> - **LLM**: Ollama (local model inference, gemma3:1b default)
> - **Workflow Automation**: n8n (webhook-triggered analysis pipelines)
> - **File Storage**: Cloudinary (PDF hosting) + local PDF parsing
> - **Communication**: REST APIs (backend ↔ frontend), HTTP (backend ↔ Ollama)

Q6: How is the frontend structured? Why React + TypeScript + Tailwind CSS?

> **Structure**: Pages (Dashboard, Gap Detection, Trends, Chatbot), Hooks (useAuth, useAnalysisHistory, useExportHistory, useSnapshots), Services (api.ts for HTTP calls), Mocks (sample data for testing).
>
> **Why these tools?**
>
> - **React**: Component reusability, efficient re-rendering, large ecosystem
> - **TypeScript**: Type safety, reduces bugs at development time, better IDE support
> - **Tailwind CSS**: Utility-first, rapid UI development, consistent design tokens, small bundle
> - **Vite**: 10x faster build than Webpack, near-instant HMR, optimized production build

---

Q7: Explain the backend Express.js architecture and how modules communicate.

> Backend uses modular architecture:
>
> - **Routes** (`src/routes/modules.js`): Expose REST endpoints (`POST /modules/1-summarization`, etc.)
> - **Services** (`src/services/`): Business logic for each module (e.g., `module6Chatbot.js`)
> - **Middleware**: Auth protection (JWT), error handling, logging
> - **Database models**: Paper, AnalysisReport, User (Mongoose schemas)
> - **Communication**: Routes orchestrate modules in sequence; each module receives papers and outputs structured results passed to next module
> - **LLM Bridge**: `pythonBridge.js` calls Python scripts; `ollamaBridge.js` posts to Ollama HTTP API

Q8: How does MongoDB store and retrieve research analysis data?

> **Collections**:
>
> - **Papers**: title, authors, year, abstract, text, keywords, embeddings
> - **AnalysisReports**: userId, papers[], modules[1-10 results], reportName, createdAt, exportedAt
> - **Users**: email, passwordHash, role, preferences

> **Retrieval**: Queries use MongoDB filters (e.g., `{ userId, createdAt: { $gt: startDate } }`) for pagination. Indexes on userId + createdAt for fast history queries. Embeddings stored for semantic search.

Core Modules & Algorithms

Q9: How does Module 2 (Topic Modeling) work? What's BERTopic's advantage over LDA?

> **BERTopic process**:
>
> 1. Extract text from all papers
> 2. Compute embeddings using BERT (pre-trained transformer)
> 3. Reduce dimensionality using UMAP
> 4. Cluster embeddings using HDBSCAN
> 5. Generate topic labels using LLM
>
> **Advantages over LDA**:
>
> - **Semantic understanding**: BERT captures context; LDA only counts word frequency
> - **Quality**: Produces more coherent, interpretable topics
> - **Speed**: Faster inference
> - **Flexibility**: Can generate custom topic labels via LLM
> - **Modern**: Leverages pre-trained deep learning vs. bag-of-words

---

Q10: Explain Module 3 (Gap Detection) — how do you identify research gaps algorithmically?

> 1. **Extract concepts**: Use named entity recognition (spaCy) to identify key entities (algorithms, datasets, metrics)
> 2. **Compute coverage**: For each concept, count how many papers mention it
> 3. **Identify unexplored combinations**: Find pairs/triples of concepts that co-occur rarely
> 4. **LLM synthesis**: Prompt Ollama to rank gaps by potential impact: "Given these papers analyze [concepts], what research questions remain unanswered?"
> 5. **Output**: Prioritized list of gaps with reasoning
>
> **Example**: If 80 papers discuss "Transformers" but only 5 discuss "Transformers + Medical Imaging", that's a gap.

Q11: Describe Module 4 (Trend Detection) — how do you detect emerging research trends?

> 1. **Temporal analysis**: Group papers by publication year
> 2. **Frequency growth**: Calculate year-over-year topic frequency increase
> 3. **Recency boost**: Weight recent topics higher
> 4. **Citation velocity**: If available, count citations per year (signals importance)
> 5. **LLM ranking**: Prompt: "Rank these topics by research momentum and potential future impact"
> 6. **Output**: Ranked list of trends with growth metrics and confidence scores
>
> **Example**: "Diffusion Models" trend from 5% (2020) → 15% (2023) → 28% (2025).

Q12: What's the purpose of Module 6 (RAG Chatbot) and how does it differ from traditional chatbots?

> **Traditional chatbot**: Uses only pre-trained LLM knowledge (static, risk of hallucinations)
>
> **RAG (Retrieval-Augmented Generation) Chatbot**:
>
> 1. User asks question
> 2. Retrieve relevant paper excerpts + analysis from database
> 3. Build context prompt: "Given these papers [context], answer: [question]"
> 4. Send to Ollama for inference
> 5. Return answer grounded in actual papers
>
> **Advantages**: Factually accurate (context-grounded), traceable (citable), domain-specific, reduces hallucinations.

LLM & Ollama Integration

Q13: Why did you choose Ollama over cloud-based LLMs? Trade-offs?

> **Why Ollama**:
>
> - **Privacy**: Papers never leave local machine (no data exfiltration risk)
> - **Cost**: Free, no API charges (vs. OpenAI $0.01-0.10 per 1k tokens)
> - **Latency**: Local inference ~1-3s (vs. cloud 5-10s + network roundtrip)
> - **Control**: Can swap models, adjust parameters, run offline
>
> **Trade-offs**:
>
> - **Quality**: Smaller models (gemma3:1b) less capable than GPT-4 or Claude
> - **Hardware**: Requires local compute (CPU/GPU)
> - **Maintenance**: Self-hosted, no SLA/support
> - **Scaling**: Harder to scale to thousands of concurrent users
>
> **Best for**: Research labs, privacy-sensitive institutions, cost-conscious teams.

Q14: How does the RAG (Retrieval-Augmented Generation) pipeline work in Module 6?

```
User Question → Extract key concepts
                  ↓
              Search papers for matching sections
                  ↓
              Rank by relevance (TF-IDF or semantic similarity)
                  ↓
              Concatenate top-k excerpts + metadata
                  ↓
              Build prompt: "Context: [papers] Question: [user query]"
                  ↓
              POST to Ollama /api/generate with prompt
```

```
                    ↓
Return streamed response with grounding
```

Key insight: LLM uses provided context, not internal knowledge, ensuring factuality.

Q15: What prompt engineering techniques are used to improve chatbot accuracy?

1. **Context framing**: "Based ONLY on these papers, answer..."
2. **Role-playing**: "You are a research analyst summarizing literature..."
3. **Chain-of-thought**: "First, identify relevant topics. Then, synthesize findings..."
4. **Example format**: "Answer in bullet points with citations"
5. **Temperature tuning**: Lower temp (0.3) for factual Q&A, higher (0.7) for creative synthesis
6. **Token limits**: Set `num_predict=256` to avoid rambling
7. **Explicit constraints**: "If you don't know, say 'Not found in papers'"

Q16: How do you handle token limits and model context windows in the chatbot?

- **Token budget**: gemma3:1b has ~2k context window
- **Strategy**: Only include top-3 most relevant papers (~500 tokens) + question (50 tokens) = ~600 tokens, leaving buffer for response (512 tokens)
- **Chunking**: If paper too long, extract abstract + most similar section to query
- **Fallback**: If context exceeds limit, prioritize recency or relevance, truncate gracefully
- **Environment variable**: `OLLAMA_CHAT_TIMEOUT_MS=30000` prevents hanging

Q17: Why is `gemma3:1b` chosen as the default model? When would you switch models?

Why gemma3:1b:

- 1B parameters: Runs on CPU/low-end GPU, fast inference (~2s)
- Good quality: Trained on 2T tokens, understands context
- Open-source: Fully transparent, auditable

When to switch:

- **Neural-chat**: Better conversational quality, slower (3-5s)
- **Mistral**: More capable reasoning, needs GPU (3-8B)
- **Zephyr**: Best instruction-following, 7B (slower)
- **llama2-chat**: Production-grade, 70B (enterprise only)

Trade-off: Model size vs. accuracy vs. speed. Choose based on hardware budget.

Data Processing

Q18: Explain the PDF parsing pipeline — how do you extract text and metadata?

Pipeline:

> 1. User uploads PDF via frontend
> 2. Save to Cloudinary (or local file)
> 3. Backend calls `pdfParser.js` (uses `pdf-parse` library)
> 4. Extract: text, metadata (title, author, pages, creation date)
> 5. Split into pages
> 6. Clean: Remove headers, footers, page numbers using regex
> 7. Store in MongoDB: { title, abstract (first 500 chars), text (full), year, authors[] }
>
> **Challenges**: Scanned PDFs (need OCR), complex layouts (tables, figures), non-English papers.

Q19: How does the text preprocessing handle different paper formats and encodings?

> 1. **Encoding detection**: Use `chardet` library to identify encoding (UTF-8, ISO-8859-1, etc.)
> 2. **Normalization**: Convert to UTF-8, remove special characters
> 3. **Tokenization**: spaCy breaks text into sentences and tokens
> 4. **Cleaning**: Remove stopwords (the, is, a), convert to lowercase
> 5. **Lemmatization**: Reduce to root form (running → run)
> 6. **Regex patterns**: Extract sections (Abstract, Introduction, Results, Conclusion)
> 7. **Fallback**: If format unrecognizable, store raw text + flag for manual review
>
> **Result**: Standardized JSON with structured sections.

Q20: What's the role of spaCy NLP in your data processing?

> **Tasks**:
>
> - **Tokenization**: Break text into meaningful units
> - **Named Entity Recognition (NER)**: Identify researchers, institutions, datasets, algorithms
> - **Part-of-speech tagging**: Distinguish nouns, verbs, adjectives
> - **Dependency parsing**: Understand sentence structure
> - **Lemmatization**: Normalize words to base forms
>
> **In ResearchLens**: Used in gap detection (extract entities), trend detection (identify key concepts), and chatbot context (semantic similarity matching).

N8N Integration

Q21: How does n8n complement ResearchLens' core functionality?

> n8n enables **low-code workflow automation**:
>
> - **Triggers**: Webhook from ResearchLens (e.g., "analysis complete")
> - **Processing**: Run custom logic (filter results, format data, send notifications)
> - **Actions**: Email reports, save to cloud, post to Slack, trigger external APIs
> - **Example workflow**: New analysis → n8n extracts gaps → sends email → posts summary to Discord
> - **Benefit**: Researchers automate repetitive tasks without coding

---

Q22: What workflows have you automated with n8n?

> 1. **Daily digest workflow**: Fetch latest papers from ArXiv → analyze gaps → email summary
> 2. **Slack notifications**: Analysis complete → post findings to research team Slack
> 3. **Report export**: Generate PDF → upload to Google Drive → share link
> 4. **Database sync**: Analysis results → backup to AWS S3
> 5. **Trend alerts**: Detect spike in specific topics → notify stakeholders

---

Q23: How do you handle webhook communication between ResearchLens and n8n?

> **Process**:
>
> 1. Backend completes analysis, calls `n8nBridge.js`
> 2. Sends POST to n8n webhook: `http://localhost:5678/webhook/{workflowId}`
> 3. Payload: `{ analysisId, userId, results: { gaps, trends, topics }, timestamp }`
> 4. n8n receives, parses, triggers actions (email, Slack, DB)
> 5. Returns confirmation: `{ status: "queued", workflowRunId: "..." }`
> 6. Frontend polls for notification or receives Slack message
>
> **Reliability**: Retry logic (max 3 attempts) for failed webhooks; fallback to message queue if n8n offline.

Performance & Scalability

Q24: What's the expected processing time for analyzing 100 papers?

> **Benchmark** (M1 Mac / i7 CPU with Ollama):
>
> - Module 1 (Summarization): 100 × 5s = 500s (500 tokens per paper)
> - Module 2 (Topic Modeling): 60s (embedding + clustering)
> - Module 3 (Gap Detection): 120s (entity extraction + LLM calls)
> - Module 4 (Trend Detection): 90s
> - Module 5 (Visualization): 30s
> - Module 6 (Chatbot): 180s
> - Module 10 (Scientific Honesty): 60s
>
> **Total**: ~1100s (~18 minutes) for 100 papers **Optimization**: Parallel processing of modules 1-5 could reduce to ~8 minutes

---

Q25: How would you scale ResearchLens to handle 10,000+ papers?

> **Strategies**:
>
> 1. **Distributed processing**: Use job queue (Bull/BullMQ) to process papers in parallel across workers
> 2. **Caching**: Store embeddings/topics so re-analyzing similar papers is instant
> 3. **GPU acceleration**: Deploy Ollama on GPU server (10x faster inference)
> 4. **Database indexing**: Add indexes on userId, createdAt, paper keywords
> 5. **Microservices**: Separate services for each module, scale independently

6. **CDN**: Cache results, serve from edge locations
7. **Pagination**: Load results in batches (100 per page) instead of all at once

Infrastructure: Kubernetes + Docker for orchestration, Kafka for queuing.

Q26: What performance bottlenecks exist and how would you address them?

Bottlenecks:

1. **Ollama inference**: Single model can't handle concurrent requests. **Fix**: Load-balance across multiple Ollama instances
2. **PDF parsing**: I/O bound. **Fix**: Async processing, queue papers
3. **Embedding computation**: CPU-intensive. **Fix**: GPU acceleration, pre-compute batches
4. **Database queries**: N+1 problem. **Fix**: Add indexes, use aggregation pipelines
5. **Memory**: Large papers consume RAM. **Fix**: Stream processing, chunk papers

Profiling: Use tools (clinic.js, py-spy) to identify actual bottlenecks before optimizing.

Q27: How do you optimize Ollama for faster inference?

1. **Quantization**: Use GGUF quantized models (4-bit) instead of float32 (2x faster, 25% quality loss)
2. **GPU utilization**: Set environment variable `CUDA_VISIBLE_DEVICES=0` to use GPU
3. **Batch inference**: Send 10 requests together instead of 1 at a time
4. **Smaller model**: Use 1B instead of 7B (5x faster)
5. **Reduce context**: Only send 200 tokens of context, not full paper
6. **Caching**: Cache identical prompts (same paper + same question)
7. **Temperature=0**: Faster decoding for deterministic (factual) tasks

---

Security & Authentication

Q28: How is user authentication handled in ResearchLens?

Flow:

1. User registers: POST `/auth/register` with email + password
2. Backend hashes password (bcrypt), stores in MongoDB
3. User logs in: POST `/auth/login` with credentials
4. Backend verifies hash, generates JWT token if valid
5. Frontend stores JWT in localStorage or httpOnly cookie
6. Frontend attaches JWT to all API requests: `Authorization: Bearer <token>`
7. Backend middleware (`auth.js`) verifies JWT signature before allowing access
8. Logout: Delete JWT (client-side); optionally blacklist on server

---

Q29: What JWT mechanisms are in place and why?

JWT Structure:

```
Header: { alg: "HS256", type: "JWT" }
Payload: { userId, email, role, iat: timestamp, exp: timestamp + 7days }
Signature: HMAC(header.payload, SECRET_KEY)
```

Why JWT:

- **Stateless**: No session database needed
- **Scalable**: Works across multiple servers
- **Secure**: Tamper-proof (signature verification)
- **Expiry**: Auto-logout after 7 days
- **Revocation**: Can blacklist tokens (e.g., on logout)

Token refresh: Issue short-lived access tokens (15 min) + long-lived refresh tokens (7 days) for better security.

Q30: How do you protect sensitive research data in the database?

1. **Field-level encryption**: Encrypt sensitive fields (e.g., notes, metadata) using AES-256
2. **Database encryption**: MongoDB Enterprise supports encrypted storage
3. **Access control**: MongoDB role-based access (users can only read their own papers)
4. **HTTPS**: All API traffic encrypted in transit
5. **Audit logging**: Log all data access (who, when, what)
6. **Secrets management**: Store API keys in `.env` file (never in code), use HashiCorp Vault in production
7. **Backup encryption**: Backups encrypted before storage

Testing & Debugging

Q31: Describe your testing strategy for the analysis modules.

1. **Unit tests**: Jest tests for each module (e.g., gap detection returns list of gaps)
2. **Integration tests**: Test full pipeline: papers → Module 1 → Module 2 → database
3. **Mock data**: Sample 5 papers with known results for consistent testing
4. **Performance tests**: Measure processing time, regression if slows >10%
5. **Quality tests**: Manual review of results (do gaps make sense? Are topics coherent?)
6. **Edge cases**: Empty papers, single word papers, non-English text
7. **Regression tests**: Re-run previous analyses, ensure consistent output

CI/CD: GitHub Actions runs tests on every commit; blocks merge if tests fail.

Q32: How do you debug issues between frontend and backend?

1. **Network debugging**: Use Postman or Insomnia to test backend endpoints directly
2. **Browser DevTools**: Check Network tab to see request/response, Console for JS errors
3. **Backend logging**: Console.log module inputs/outputs; structured logging (winston)
4. **Database inspection**: MongoDB Compass to view stored data, verify correctness

5. **Stack traces**: Full error messages with line numbers
6. **State inspection**: Frontend Redux DevTools to replay state changes
7. **Request/response**: Log full payloads to identify field mismatches

Example: If chatbot returns empty answer, check: (1) Ollama running? (2) Papers in DB? (3) Backend log has LLM response?

Q33: What monitoring/logging systems do you have in place?

- **Backend logging**: Winston/Morgan logs HTTP requests (status, latency, user)
- **Error tracking**: Sentry captures uncaught errors with stack traces
- **Performance monitoring**: Custom timers log module execution time
- **Database monitoring**: MongoDB Atlas alerts on slow queries
- **Uptime monitoring**: Ping backend health endpoint every 5 min; alert if down
- **Logs storage**: ELK Stack (Elasticsearch, Logstash, Kibana) for centralized log search
- **Metrics**: Prometheus exports metrics (requests/sec, latency, error rate) for Grafana dashboards

---

Deployment & DevOps

Q34: How would you deploy ResearchLens to production?

Steps:

1. **Build frontend**: `npm run build` → optimized `dist/` folder
2. **Build backend**: `npm run build` (if using TypeScript); ensure all tests pass
3. **Environment setup**: Create production `.env` (real MongoDB URI, production API keys)
4. **Docker**: Build Docker images for frontend + backend
5. **Registry**: Push to Docker Hub or private ECR
6. **Orchestration**: Deploy on Kubernetes (or Docker Compose on single server)
7. **Database**: Provision MongoDB Atlas (managed service) or self-hosted replica set
8. **Reverse proxy**: Nginx/Caddy routes requests to frontend/backend containers
9. **HTTPS**: Let's Encrypt SSL certificates, auto-renewal
10. **Monitoring**: Prometheus + Grafana for observability
11. **CD**: GitHub Actions auto-deploys on git push to `main` branch

---

Q35: What are the hardware requirements for running Ollama at scale?

For gemma3:1b:

- **Minimum**: 4GB RAM, 2-core CPU, 30s inference per request
- **Recommended**: 8GB RAM, 4-core CPU, ~2s inference
- **With GPU** (NVIDIA): 6GB VRAM, <1s inference (5-10x faster)

Scaling:

- **10 concurrent users**: 1 × 8GB machine with GPU
- **100 concurrent users**: 2 × machines load-balanced
- **1000+ concurrent users**: Kubernetes cluster with auto-scaling

---

Q36: How would you containerize ResearchLens (Docker)?

> **Dockerfiles**:
>
> ```
> # Backend Dockerfile
> FROM node:18-alpine
> WORKDIR /app
> COPY package*.json ./
> RUN npm ci --production
> COPY . .
> CMD ["npm", "run", "dev"]
> EXPOSE 4000
>
> # Frontend Dockerfile
> FROM node:18-alpine AS build
> WORKDIR /app
> COPY package*.json ./
> RUN npm ci
> COPY . .
> RUN npm run build
>
> FROM nginx:alpine
> COPY --from=build /app/dist /usr/share/nginx/html
> COPY nginx.conf /etc/nginx/conf.d/default.conf
> EXPOSE 80
> ```
>
> **docker-compose.yml**: Orchestrate frontend, backend, MongoDB, Ollama services with networking.

Q37: What's your backup and disaster recovery strategy?

> **Backup**:
>
> - **Database**: Daily automated backup to AWS S3 (encryption at rest)
> - **Code**: GitHub repo (redundant remote)
> - **User data**: Backups retained for 30 days
>
> **Disaster recovery**:
>
> - **RTO** (Recovery Time Objective): 1 hour (restore from backup, restart services)
> - **RPO** (Recovery Point Objective): 24 hours (lose at most 1 day of data)
> - **Failover**: DNS switches to secondary server if primary down
> - **Testing**: Monthly restore drills to ensure backups work
> - **Documentation**: Runbooks for common failure scenarios

Project Challenges & Solutions

Q38: What was the biggest technical challenge during development?

> **Challenge**: Integrating Python (gap/trend detection) with Node.js backend while maintaining performance and reliability.
>
> **Problem**: Python scripts slow to start (cold boot ~2s), errors kill process.
>
> **Solution**:
>
> - Used `pythonBridge.js` to spawn child processes with error handling
> - Implemented timeouts (30s max) to prevent hanging
> - Added retry logic (max 3 attempts) for transient failures
> - Cached Python results in MongoDB so repeated analyses don't re-run
> - Considered alternatives: calling Python via REST (Py server), but settled on child process for lower latency
>
> **Lesson**: Different languages have different strengths; bridge them gracefully with clear error handling.

Q39: How did you solve the Python-Node.js bridge communication issue?

> **Approaches considered**:
>
> 1. ~~Child process (exec)~~ → Slow startup, no streaming
> 2. ~~REST server~~ → Adds complexity, network latency
> 3. **Chosen: Child process (spawn) with stdin/stdout**
>
> ```
> const pythonProcess = spawn('python', ['gap_detection_cli.py', papersJSON]);
> pythonProcess.stdout.on('data', (data) => {
>   const result = JSON.parse(data);
>   res.json(result);
> });
> pythonProcess.stderr.on('data', (err) => console.error(err));
> ```
>
> **Benefits**: Simple, no extra servers, JSON I/O, error handling via stderr. **Drawback**: Spawning new process per request is slow; mitigated by results caching.

Q40: Explain how you handled the Ollama CLI PATH issues on Windows.

> **Problem**: Installed Ollama but `ollama` command not found in PowerShell.
>
> **Root cause**: Ollama installer didn't add `C:\Users\<user>\AppData\Local\Programs\Ollama` to PATH.
>
> **Solutions**:
>
> 1. **Short-term**: Use HTTP API directly (`http://localhost:11434/api/generate`) — works without CLI
> 2. **Medium-term**: Add Ollama to PATH manually:

```
$env:PATH += ";C:\Users\<user>\AppData\Local\Programs\Ollama"
```

3. **Long-term**: Reinstall Ollama with PATH option checked (or use WSL2 on Windows)

Lesson: Cloud-native tools (like Ollama) can be accessed via HTTP API even if CLI not in PATH — reduces dev friction.

Scientific Honesty Module

Q41: How do you assess scientific honesty/bias in research papers?

Heuristics:

1. **Conflicts of interest**: Check if authors affiliated with company that benefits from results
2. **Reproducibility**: Does paper provide code, data, hyperparameters?
3. **Evaluation**: Multiple datasets tested? Comparison with baselines? Error bars reported?
4. **Statistical significance**: P-values reported? Effect sizes?
5. **Negation of results**: Does abstract match conclusions? Any cherry-picked results?
6. **Limitations**: Honest discussion of method limitations?
7. **Funding disclosure**: Is funding source disclosed?

Output: Honesty score (0-100), flags for each criterion.

Note: This is heuristic-based; real bias detection requires human expert review.

Business & Impact

Q42: What's the target user base for ResearchLens?

1. **PhD students**: Struggling with literature reviews; need to synthesize 100+ papers
2. **Postdocs**: Rapidly survey new research directions
3. **Industry researchers**: Competitive intelligence, tracking ML innovations
4. **Grant reviewers**: Quickly assess research landscape for grant decisions
5. **Subject matter experts**: Summarize interdisciplinary findings
6. **Librarians**: Help patrons navigate large corpora
7. **Institutions**: Bulk analysis of institutional research output

Secondary: Journals (to peer-review literature reviews), funders (to identify funding priorities).

Q43: How does ResearchLens save researchers time compared to manual review?

Time comparison (100 papers):

Task	Manual	ResearchLens
Reading papers	40 hours	0 (auto-extracted)

Task	Manual	ResearchLens
Identifying topics	4 hours	30 sec
Finding gaps	8 hours	2 min (LLM-assisted)
Detecting trends	4 hours	1 min
Writing related work	6 hours	5 min (skeleton)
Total	**62 hours**	**10 minutes**

Savings: 62 hours → 10 min (0.16 hours), 99.7% time reduction. **Cost**: At $50/hour research wage, saves $3,100 per analysis.

Q44: What's the commercial potential of this tool?

Revenue streams:

1. **SaaS model**: $50-200/month subscription (individuals); $500-2000/month (institutions)
2. **API licensing**: $100-500/month for journals/platforms to embed analysis
3. **Enterprise**: Custom deployment + support for large organizations ($10k+/year)
4. **White-label**: Allow universities to rebrand for internal use

Market size:

- ~50k active PhD students writing dissertations annually
- Each spends $2-5k on research tools
- Market: $100-250M TAM

Competitive advantage: Open-source backend (Ollama) = lower costs than commercial AI tools.

Q45: How could ResearchLens be integrated into academic institutions?

Integration points:

1. **University library systems**: Researchers access via institution portal
2. **Learning management systems (Canvas, Blackboard)**: Integrated as plugin
3. **Institutional repositories**: Auto-analyze all papers submitted
4. **Grant management systems**: Analyze funded research, identify gaps
5. **Peer review workflows**: Help reviewers quickly assess paper quality
6. **Curriculum planning**: Identify emerging topics to include in courses
7. **Research assessment**: Institutional reporting on research impact/trends

Deployment: Private cloud (on-premises) for data sovereignty, federated learning to anonymize cross-institution insights.

Future Enhancements

Q46: What features would you add to ResearchLens next?

1. **Real-time collaboration**: Multiple researchers analyze same corpus together
2. **Multi-language support**: Analyze papers in Chinese, Spanish, French, etc.
3. **Graph visualization**: Interactive knowledge graph showing paper relationships
4. **Citation network analysis**: Identify most-cited papers and citation patterns
5. **Author network analysis**: Collaborations, geographic distribution
6. **PDF highlighting**: Frontend overlay highlighting key sections
7. **Fact-checking**: Cross-validate claims across papers (detect contradictions automatically)
8. **Integration with external APIs**: ArXiv, PubMed, Semantic Scholar to auto-fetch papers
9. **Advanced NLP**: Relation extraction (Paper A cites Paper B as foundation for method C)
10. **Customizable modules**: Let users write custom analysis logic

---

Q47: How could multi-language support be implemented?

1. **PDF parsing**: OCR for scanned PDFs in any language
2. **Translation**: Translate non-English papers to English (or keep multilingual)
3. **NLP models**: Use multilingual BERT (`bert-base-multilingual-cased`) for embeddings
4. **Tokenization**: spaCy supports 17+ languages (French, German, Spanish, etc.)
5. **UI localization**: Frontend supports i18n (translations for UI strings)
6. **LLM prompts**: Adapt prompts for language (e.g., prompt gemma3 to respond in German if asked)

Challenge: Not all LLMs equally strong in all languages; may need language-specific model selection.

Q48: Would you consider adding real-time collaborative analysis?

Design:

1. **WebSocket connection**: Multiple users connect to same analysis session
2. **Shared cursor**: See what section others are viewing
3. **Comments/annotations**: Highlight text + leave notes (like Google Docs)
4. **Version history**: Track who made what edits when
5. **Permissions**: Read-only (reviewers) vs. edit (team)
6. **Conflict resolution**: Last-write-wins or operational transformation (OT)

Implementation: Socket.io (WebSocket library) + Redux for state sync. **Challenge**: Real-time sync can be complex; consider using Yjs (CRDT library) for conflict-free collaboration.

Q49: How could you integrate with academic databases (PubMed, ArXiv, etc.)?

Integration approach:

1. **API calls**: Use PubMed API, ArXiv API to search/fetch papers
2. **Webhook triggers**: On new paper published, auto-download and analyze
3. **Scheduled jobs**: Cron job runs every night, fetches new papers matching user query
4. **Data pipeline**: Download → Parse → Analyze → Store in MongoDB
5. **Feed UI**: Show latest analyses in dashboard feed

Example workflow:

```
User searches "machine learning" on ArXiv
→ Backend calls ArXiv API (returns 1000 papers)
→ Queue analysis for all 1000 (using Bull job queue)
→ Results stream in as they complete (WebSocket)
→ User exports results as CSV
```

APIs to use:

- **ArXiv**: `http://export.arxiv.org/api/query?search_query=cat:cs.AI`
- **PubMed**: `https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi`
- **Semantic Scholar**: GraphQL API for structured data
- **Unpaywall**: Free access to open-access PDFs

Viva Tips & Best Practices

✅ **Do**:

- Know your architecture inside-out
- Practice explaining complex concepts simply
- Have specific code examples ready
- Acknowledge limitations honestly
- Discuss trade-offs thoughtfully
- Show confidence in your work

❌ **Don't**:

- Memorize answers word-for-word (sounds robotic)
- Pretend to know something you don't (say "I'll research that")
- Over-explain (concise is better)
- Get defensive about criticism
- Use jargon without explanation

Structure your answers:

1. **Brief answer** (1 sentence)
2. **Explanation** (3-4 sentences with example)
3. **Trade-offs** (what else you considered)
4. **Future improvements** (what you'd do differently)

Good luck with your viva! 🎓